ChatGLM3 6B (ChatGLM3 Team, 2023), as well as models with enhancements in mathematics including (5) InternLM2-Math 20B [6] which builds on InternLM2 and underwent math training followed by instruction tuning, (6) Math-Shepherd-Mistral 7B which applys PPO training (Schulman et al., 2017) to Mistral 7B (Jiang et al., 2023) with a process-supervised reward model, (7) the WizardMath series (Luo et al., 2023) which improves mathematical reasoning in Mistral 7B and Llama-2 70B (Touvron et al., 2023) using evolve-instruct (i.e., a version of instruction tuning that uses AI-evolved instructions) and PPO training with training problems primarily sourced from GSM8K and MATH, (8) MetaMath 70B (Yu et al., 2023) which is Llama-2 70B fine-tuned on an augmented version of GSM8K and MATH, (9) ToRA 34B Gou et al. (2023) which is CodeLlama 34B fine-tuned to do tool-integrated mathematical reasoning, (10) MAmmoTH 70B (Yue et al., 2023) which is Llama-2 70B instruction-tuned on MathInstruct.

As shown in Table 5, under the evaluation setting where tool use is disallowed, DeepSeekMath-Instruct 7B demonstrates strong performance of step-by-step reasoning. Notably, on the competition-level MATH dataset, our model surpasses all open-source models and the majority of proprietary models (e.g., Inflection-2 and Gemini Pro) by at least 9% absolute. This is true even for models that are substantially larger (e.g., Qwen 72B) or have been specifically enhanced through math-focused reinforcement learning (e.g., WizardMath-v1.1 7B). While DeepSeekMath-Instruct rivals the Chinese proprietary models GLM-4 and Baichuan-3 on MATH, it still underperforms GPT-4 and Gemini Ultra.

Under the evaluation setting where models are allowed to integrate natural language reasoning and program-based tool use for problem solving, DeepSeekMath-Instruct 7B approaches an accuracy of 60% on MATH, surpassing all existing open-source models. On the other benchmarks, our model is competitive with DeepSeek-LLM-Chat 67B, the prior state-of-the-art that is 10 times larger.

4. Reinforcement Learning

4.1. Group Relative Policy Optimization

Reinforcement learning (RL) has been proven to be effective in further improving the mathematical reasoning ability of LLMs after the Supervised Fine-Tuning (SFT) stage (Luo et al., 2023; Wang et al., 2023b). In this section, we introduce our efficient and effective RL algorithm, Group Relative Policy Optimization (GRPO).

4.1.1. From PPO to GRPO

Proximal Policy Optimization (PPO) (Schulman et al., 2017) is an actor-critic RL algorithm that is widely used in the RL fine-tuning stage of LLMs (Ouyang et al., 2022). In particular, it optimizes LLMs by maximizing the following surrogate objective:

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})}, 1-\varepsilon, 1+\varepsilon \right) A_t \right], \quad (1)$$

where π_θ and $\pi_{\theta_{old}}$ are the current and old policy models, and q, o are questions and outputs sampled from the question dataset and the old policy $\pi_{\theta_{old}}$, respectively. ε is a clipping-related hyper-parameter introduced in PPO for stabilizing training. A_t is the advantage, which is computed by applying Generalized Advantage Estimation (GAE) (Schulman et al., 2015), based

[6] https://github.com/InternLM/InternLM-Math

Model	Size	English Benchmarks		Chinese Benchmarks	
		GSM8K	MATH	MGSM-zh	CMATH
Chain-of-Thought Reasoning					
Closed-Source Model					
Gemini Ultra	-	94.4%	53.2%	-	-
GPT-4	-	92.0%	52.9%	-	86.0%
Inflection-2	-	81.4%	34.8%	-	-
GPT-3.5	-	80.8%	34.1%	-	73.8%
Gemini Pro	-	86.5%	32.6%	-	-
Grok-1	-	62.9%	23.9%	-	-
Baichuan-3	-	88.2%	49.2%	-	-
GLM-4	-	87.6%	47.9%	-	-
Open-Source Model					
InternLM2-Math	20B	82.6%	37.7%	-	-
Qwen	72B	78.9%	35.2%	-	-
Math-Shepherd-Mistral	7B	84.1%	33.0%	-	-
WizardMath-v1.1	7B	83.2%	33.0%	-	-
DeepSeek-LLM-Chat	67B	84.1%	32.6%	74.0%	80.3%
MetaMath	70B	82.3%	26.6%	66.4%	70.9%
SeaLLM-v2	7B	78.2%	27.5%	64.8%	-
ChatGLM3	6B	72.3%	25.7%	-	-
WizardMath-v1.0	70B	81.6%	22.7%	64.8%	65.4%
DeepSeekMath-Instruct	7B	82.9%	46.8%	73.2%	84.6%
DeepSeekMath-RL	7B	**88.2%**	**51.7%**	**79.6%**	**88.8%**
Tool-Integrated Reasoning					
Closed-Source Model					
GPT-4 Code Interpreter	-	97.0%	69.7%	-	-
Open-Source Model					
InternLM2-Math	20B	80.7%	54.3%	-	-
DeepSeek-LLM-Chat	67B	86.7%	51.1%	76.4%	85.4%
ToRA	34B	80.7%	50.8%	41.2%	53.4%
MAmmoTH	70B	76.9%	41.8%	-	-
DeepSeekMath-Instruct	7B	83.7%	57.4%	72.0%	84.3%
DeepSeekMath-RL	7B	**86.7%**	**58.8%**	**78.4%**	**87.6%**

Table 5 | Performance of Open- and Closed-Source models with both Chain-of-Thought and Tool-Integrated Reasoning on English and Chinese Benchmarks. Scores in gray denote majority votes with 32 candidates; The others are Top1 scores. DeepSeekMath-RL 7B beats all open-source models from 7B to 70B, as well as the majority of closed-source models. Although DeepSeekMath-RL 7B is only further trained on chain-of-thought-format instruction tuning data of GSM8K and MATH, it improves over DeepSeekMath-Instruct 7B on all benchmarks.

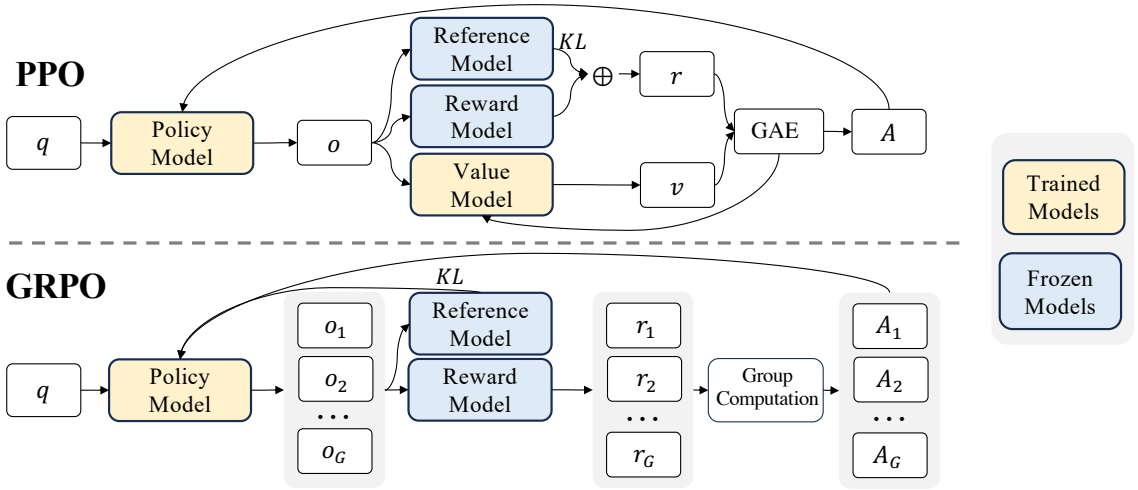


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

on the rewards $\{r_{\geq t}\}$ and a learned value function V_ψ. Thus, in PPO, a value function needs to be trained alongside the policy model and to mitigate over-optimization of the reward model, the standard approach is to add a per-token KL penalty from a reference model in the reward at each token (Ouyang et al., 2022), i.e.,

$$r_t = r_\varphi(q, o_{\leq t}) - \beta \log \frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})}, \tag{2}$$

where r_φ is the reward model, π_{ref} is the reference model, which is usually the initial SFT model, and β is the coefficient of the KL penalty.

As the value function employed in PPO is typically another model of comparable size as the policy model, it brings a substantial memory and computational burden. Additionally, during RL training, the value function is treated as a baseline in the calculation of the advantage for variance reduction. While in the LLM context, usually only the last token is assigned a reward score by the reward model, which may complicate the training of a value function that is accurate at each token. To address this, as shown in Figure 4, we propose Group Relative Policy Optimization (GRPO), which obviates the need for additional value function approximation as in PPO, and instead uses the average reward of multiple sampled outputs, produced in response to the same question, as the baseline. More specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_{i,t}\right] - \beta\mathbb{D}_{KL}\left[\pi_\theta||\pi_{ref}\right]\right\}, \tag{3}$$

where ε and β are hyper-parameters, and $\hat{A}_{i,t}$ is the advantage calculated based on relative rewards of the outputs inside each group only, which will be detailed in the following subsections. The group relative way that GRPO leverages to calculate the advantages, aligns well with the comparative nature of rewards models, as reward models are typically trained on datasets of comparisons between outputs on the same question. Also note that, instead of adding KL penalty in the reward, GRPO regularizes by directly adding the KL divergence between the trained policy and the reference policy to the loss, avoiding complicating the calculation of $\hat{A}_{i,t}$.

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_φ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for iteration = 1, ..., I **do**
  reference model $\pi_{ref} \leftarrow \pi_\theta$
  for step = 1, ..., M **do**
    Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
    Update the old policy model $\pi_{\theta_{old}} \leftarrow \pi_\theta$
    Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
    Compute rewards $\{r_i\}_{i=1}^{G}$ for each sampled output o_i by running r_φ
    Compute $\hat{A}_{i,t}$ for the t-th token of o_i through group relative advantage estimation.
    for GRPO iteration = 1, ..., μ **do**
      Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
  Update r_φ through continuous training using a replay mechanism.

Output π_θ

And different from the KL penalty term used in (2), we estimate the KL divergence with the following unbiased estimator (Schulman, 2020):

$$\mathbb{D}_{KL}\left[\pi_\theta||\pi_{ref}\right] = \frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - \log\frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - 1, \tag{4}$$

which is guaranteed to be positive.

4.1.2. Outcome Supervision RL with GRPO

Formally, for each question q, a group of outputs $\{o_1, o_2, \cdots, o_G\}$ are sampled from the old policy model $\pi_{\theta_{old}}$. A reward model is then used to score the outputs, yielding G rewards $\mathbf{r} = \{r_1, r_2, \cdots, r_G\}$ correspondingly. Subsequently, these rewards are normalized by subtracting the group average and dividing by the group standard deviation. Outcome supervision provides the normalized reward at the end of each output o_i and sets the advantages $\hat{A}_{i,t}$ of all tokens in the output as the normalized reward, i.e., $\hat{A}_{i,t} = \widetilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$, and then optimizes the policy by maximizing the objective defined in equation (3).

4.1.3. Process Supervision RL with GRPO

Outcome supervision only provides a reward at the end of each output, which may not be sufficient and efficient to supervise the policy in complex mathematical tasks. Following Wang et al. (2023b), we also explore process supervision, which provides a reward at the end of each reasoning step. Formally, given the question q and G sampled outputs $\{o_1, o_2, \cdots, o_G\}$, a process reward model is used to score each step of the outputs, yielding corresponding rewards: $\mathbf{R} = \{\{r_1^{index(1)}, \cdots, r_1^{index(K_1)}\}, \cdots, \{r_G^{index(1)}, \cdots, r_G^{index(K_G)}\}\}$, where $index(j)$ is the end token index of the j-th step, and K_i is the total number of steps in the i-th output. We also normalize these rewards with the average and the standard deviation, i.e., $\widetilde{r}_i^{index(j)} = \frac{r_i^{index(j)} - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})}$. Subsequently, the process supervision calculates the advantage of each token as the sum of the normalized rewards from the following steps, i.e., $\hat{A}_{i,t} = \sum_{index(j)\geq t} \widetilde{r}_i^{index(j)}$, and then optimizes the policy by maximizing the objective defined in equation (3).

4.1.4. Iterative RL with GRPO

As the reinforcement learning training process progresses, the old reward model may not be sufficient to supervise the current policy model. Therefore, we also explore the iterative RL with GRPO. As shown in Algorithm 1, in iterative GRPO, we generate new training sets for the reward model based on the sampling results from the policy model and continually train the old reward model using a replay mechanism that incorporates 10% of historical data. Then, we set the reference model as the policy model, and continually train the policy model with the new reward model.

4.2. Training and Evaluating DeepSeekMath-RL

We conduct RL based on DeepSeekMath-Instruct 7B. The training data of RL are chain-of-thought-format questions related to GSM8K and MATH from the SFT data, which consists of around 144K questions. We exclude other SFT questions to investigate the impact of RL on benchmarks that lack data throughout the RL phase. We construct the training set of reward models following (Wang et al., 2023b). We train our initial reward model based on the DeepSeekMath-Base 7B with a learning rate of 2e-5. For GRPO, we set the learning rate of the policy model as 1e-6. The KL coefficient is 0.04. For each question, we sample 64 outputs. The max length is set to 1024, and the training batch size is 1024. The policy model only has a single update following each exploration stage. We evaluate DeepSeekMath-RL 7B on benchmarks following DeepSeekMath-Instruct 7B. For DeepSeekMath-RL 7B, GSM8K and MATH with chain-of-thought reasoning can be regarded as in-domain tasks and all the other benchmarks can be regarded as out-of-domain tasks.

Table 5 demonstrates the performance of open- and closed-source models with both chain-of-thought and tool-integrated reasoning on English and Chinese benchmarks. We find that: 1) DeepSeekMath-RL 7B attains accuracies of 88.2% and 51.7% on GSM8K and MATH, respectively, utilizing chain-of-thought reasoning. This performance surpasses that of all open-source models in the 7B to 70B range, as well as the majority of closed-source models. 2) Crucially, DeepSeekMath-RL 7B is only trained on chain-of-thought-format instruction tuning data of GSM8K and MATH, starting from DeepSeekMath-Instruct 7B. Despite the constrained scope of its training data, it outperforms DeepSeekMath-Instruct 7B across all evaluation metrics, showcasing the effectiveness of reinforcement learning.

5. Discussion

In this section, we will share our findings in pre-training and RL experiments.

5.1. Lessons Learnt in Pre-Training

We first share our experience in pre-training. Unless otherwise specified, we will adhere to the training settings outlined in Section 2.2.1. It is worth noting that, when referring to the DeepSeekMath Corpus in this section, we use an 89B-token dataset from the second iteration of the data collection process.

5.1.1. Code Training Benefits Mathematical Reasoning

A popular yet unverified hypothesis suggests that code training improves reasoning. We attempt to offer a partial response to this, particularly within the mathematical domain: code training

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

arXiv:2501.12948v2 [cs.CL] 4 Jan 2026

Abstract

General reasoning represents a long-standing and formidable challenge in artificial intelligence. Recent breakthroughs, exemplified by large language models (LLMs) (Brown et al., 2020; OpenAI, 2023) and chain-of-thought prompting (Wei et al., 2022b), have achieved considerable success on foundational reasoning tasks. However, this success is heavily contingent upon extensive human-annotated demonstrations, and models' capabilities are still insufficient for more complex problems. Here we show that the reasoning abilities of LLMs can be incentivized through pure reinforcement learning (RL), obviating the need for human-labeled reasoning trajectories. The proposed RL framework facilitates the emergent development of advanced reasoning patterns, such as self-reflection, verification, and dynamic strategy adaptation. Consequently, the trained model achieves superior performance on verifiable tasks such as mathematics, coding competitions, and STEM fields, surpassing its counterparts trained via conventional supervised learning on human demonstrations. Moreover, the emergent reasoning patterns exhibited by these large-scale models can be systematically harnessed to guide and enhance the reasoning capabilities of smaller models.

1. Introduction

Reasoning capability, the cornerstone of human intelligence, enables complex cognitive tasks ranging from mathematical problem-solving to logical deduction and programming. Recent advances in artificial intelligence have demonstrated that large language models (LLMs) can exhibit emergent behaviors, including reasoning abilities, when scaled to a sufficient size (Kaplan et al., 2020; Wei et al., 2022a). However, achieving such capabilities in pre-training typically demands substantial computational resources. In parallel, a complementary line of research has demonstrated that large language models can be effectively augmented through chain-of-thought (CoT) prompting. This technique, which involves either providing carefully designed few-shot examples or using minimalistic prompts such as "Let's think step by step"(Kojima et al., 2022; Wei et al., 2022b), enables models to produce intermediate reasoning steps, thereby substantially enhancing their performance on complex tasks. Similarly, further performance gains have been observed when models learn high-quality, multi-step reasoning trajectories during the post-training phase (Chung et al., 2024; OpenAI, 2023). Despite their effectiveness, these approaches exhibit notable limitations. Their dependence on human-annotated reasoning traces hinders scalability and introduces cognitive biases. Furthermore, by constraining models to replicate human thought processes, their performance is inherently capped by the human-

provided exemplars, which prevents the exploration of superior, non-human-like reasoning pathways.

To tackle these issues, we aim to explore the potential of LLMs for developing reasoning abilities through self-evolution in an RL framework, with minimal reliance on human labeling efforts. Specifically, we build upon DeepSeek-V3-Base (DeepSeek-AI, 2024b) and employ Group Relative Policy Optimization (GRPO) (Shao et al., 2024) as our RL framework. The reward signal is solely based on the correctness of final predictions against ground-truth answers, without imposing constraints on the reasoning process itself. Notably, we bypass the conventional supervised fine-tuning (SFT) phase before RL training. This design choice stems from our hypothesis that human-defined reasoning patterns may limit model exploration, whereas unrestricted RL training can better incentivize the emergence of novel reasoning capabilities in LLMs. Through this process, detailed in Section 2, our model (referred to as DeepSeek-R1-Zero) naturally developed diverse and sophisticated reasoning behaviors. In solving reasoning problems, the model exhibits a tendency to generate longer responses, incorporating verification, reflection, and the exploration of alternative approaches within each response. Although we do not explicitly teach the model how to reason, it successfully learns improved reasoning strategies through reinforcement learning.

Although DeepSeek-R1-Zero demonstrates excellent reasoning capabilities, it faces challenges such as poor readability and language mixing, occasionally combining English and Chinese within a single chain-of-thought response. Furthermore, the rule-based RL training stage of DeepSeek-R1-Zero is narrowly focused on reasoning tasks, resulting in limited performance in broader areas such as writing and open-domain question answering. To address these challenges, we introduce DeepSeek-R1, a model trained through a multi-stage learning framework that integrates rejection sampling, reinforcement learning, and supervised fine-tuning, detailed in Section 3. This training pipeline enables DeepSeek-R1 to inherit the reasoning capabilities of its predecessor, DeepSeek-R1-Zero, while aligning model behavior with human preferences through additional non-reasoning data.

To enable broader access to powerful AI at a lower energy cost, we have distilled several smaller models and made them publicly available. These distilled models exhibit strong reasoning capabilities, surpassing the performance of their original instruction-tuned counterparts. We believe that these instruction-tuned versions will also significantly contribute to the research community by providing a valuable resource for understanding the mechanisms underlying long chain-of-thought (CoT) reasoning models and for fostering the development of more powerful reasoning models. We release DeepSeek-R1 series models to the public at `https://huggingface.co/deepseek-ai`.

2. DeepSeek-R1-Zero

We begin by elaborating on the training of DeepSeek-R1-Zero, which relies exclusively on reinforcement learning without supervised fine-tuning. To facilitate large-scale RL efficiency, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024).

2.1. Group Relative Policy Optimization

GRPO (Shao et al., 2024) is the reinforcement learning algorithm that we adopt to train DeepSeek-R1-Zero and DeepSeek-R1. It was originally proposed to simplify the training process and reduce the resource consumption of Proximal Policy Optimization (PPO) (Schulman et al., 2017), which is widely used in the RL stage of LLMs (Ouyang et al., 2022).

For each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \tag{1}$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \tag{2}$$

where π_{ref} is a reference policy, ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}. \tag{3}$$

We give a comparison of GRPO and PPO in Supplementary A.3. To train DeepSeek-R1-Zero, we set the learning rate to 3e-6, the KL coefficient to 0.001, and the sampling temperature to 1 for rollout. For each question, we sample 16 outputs with a maximum length of 32,768 tokens before the 8.2k step and 65,536 tokens afterward. As a result, both the performance and response length of DeepSeek-R1-Zero exhibit a significant jump at the 8.2k step, with training continuing for a total of 10,400 steps, corresponding to 1.6 training epochs. Each training step consists of 32 unique questions, resulting in a training batch size of 512. Every 400 steps, we replace the reference model with the latest policy model. To accelerate training, each rollout generates 8,192 outputs, which are randomly split into 16 mini-batches and trained for only a single inner epoch.

Table 1 | Template for DeepSeek-R1-Zero. prompt will be replaced with the specific reasoning question during training.

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>...</think>` and `<answer>...</answer>` tags, respectively, i.e., `<think>` reasoning process here `</think>` `<answer>` answer here `</answer>`. User: prompt. Assistant:

Our high-performance RL infrastructure is described in Supplementary B.1, ensuring scalable and efficient training.

2.2. Reward Design

The reward is the source of the training signal, which decides the direction of RL optimization. For DeepSeek-R1-Zero, we employ rule-based rewards to deliver precise feedback for data in mathematical, coding, and logical reasoning domains. Our rule-based reward system mainly consists of two types of rewards: accuracy rewards and format rewards.

Accuracy rewards evaluate whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for code competition prompts, a compiler can be utilized to evaluate the model's

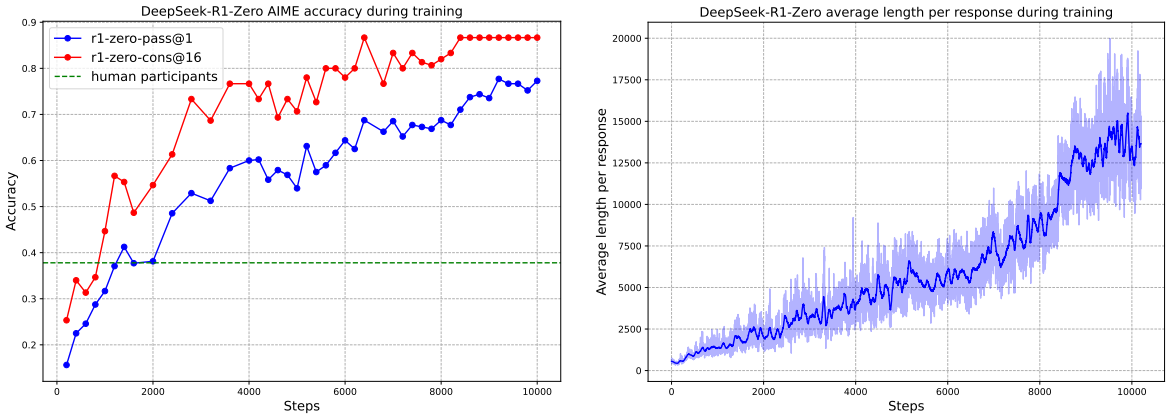


Figure 1 | (a) AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Table 32. Pass@1 and Cons@16 are described in Supplementary D.1. The baseline is the average score achieved by human participants in the AIME competition. (b) The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time. Note that a training step refers to a single policy update operation.

responses against a suite of predefined test cases, thereby generating objective feedback on correctness.

Format rewards complement the accuracy reward model by enforcing specific formatting requirements. In particular, the model is incentivized to encapsulate its reasoning process within designated tags, specifically '`<think>`' and '`</think>`'. This ensures that the model's thought process is explicitly delineated, enhancing interpretability and facilitating subsequent analysis.

$$Reward_{\text{rule}} = Reward_{\text{acc}} + Reward_{\text{format}} \tag{4}$$

The accuracy, reward and format reward are combined with the same weight. Notably, we abstain from applying neural reward models—whether outcome-based or process-based—to reasoning tasks. This decision is predicated on our observation that neural reward models are susceptible to reward hacking during large-scale reinforcement learning. Moreover, retraining such models necessitates substantial computational resources and introduces additional complexity into the training pipeline, thereby complicating the overall optimization process.

2.3. Incentivize Reasoning Capability in LLMs

Specifically, we apply the RL technique on the DeepSeek-V3 base to train DeepSeek-R1-Zero. During training, we design a straightforward template, to require DeepSeek-R1-Zero to first produce a reasoning process, followed by the final answer. We intentionally limit our constraints to this structural format, avoiding any content-specific biases to ensure that we can accurately observe the model's natural progression during the RL process.

Figure 1(a) depicts the performance trajectory of DeepSeek-R1-Zero on the AIME 2024 benchmark throughout the RL training process, where the average pass@1 score on AIME 2024 shows a significant increase, jumping from an initial 15.6% to 77.9%. In addition, by leveraging the self-consistency decoding (Wang et al., 2023c), the model's performance can be

Table 2 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a + x}} = x$ First, let's square both sides: $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

further improved, achieving an accuracy of 86.7%. This performance significantly surpasses the average performance across all human competitors. Besides the math competitions, as shown in Figure 10, DeepSeek-R1-Zero also achieves remarkable performance in coding competitions and graduate-level biology, physics, and chemistry problems. These results underscore the effectiveness of RL in enhancing the reasoning capabilities of large language models.

The self-evolution of DeepSeek-R1-Zero exemplifies how RL can autonomously enhance a model's reasoning capabilities.

As shown in Figure 1(b), DeepSeek-R1-Zero exhibits a steady increase in thinking time throughout training, driven solely by intrinsic adaptation rather than external modifications. Leveraging long CoT, the model progressively refines its reasoning, generating hundreds to thousands of tokens to explore and improve its problem-solving strategies.

The increase in thinking time fosters the autonomous development of sophisticated behaviors. Specifically, DeepSeek-R1-Zero increasingly exhibits advanced reasoning strategies such as reflective reasoning and systematic exploration of alternative solutions (see Figure 9(a) in Supplementary C.2 for details), significantly boosting its performance on verifiable tasks like math and coding. Notably, during training, DeepSeek-R1-Zero exhibits an "aha moment" (Table 2), characterized by a sudden increase in the use of the word "wait" during reflections (see Figure 9(b) in Supplementary C.2 for details). This moment marks a distinct change in reasoning patterns and clearly shows the self-evolution process of DeepSeek-R1-Zero.

The self-evolution of DeepSeek-R1-Zero underscores the power and beauty of RL: rather than explicitly teaching the model how to solve a problem, we simply provide it with the right incentives, and it autonomously develops advanced problem-solving strategies. This serves as a reminder of the potential of RL to unlock higher levels of capabilities in LLMs, paving the way for more autonomous and adaptive models in the future.

Cognitive Behaviors that Enable Self-Improving Reasoners, *or,* Four Habits of Highly Effective STaRs

Kanishk Gandhi
Stanford University

Ayush Chakravarthy
Stanford University

Anikait Singh
Stanford University

Nathan Lile
SynthLabs

Noah D. Goodman
Stanford University

arXiv:2503.01307v2 [cs.CL] 15 Aug 2025

Abstract

Test-time inference has emerged as a powerful paradigm for enabling language models to "think" longer and more carefully about complex challenges, much like skilled human experts. While reinforcement learning (RL) can drive self-improvement in language models on verifiable tasks, some models exhibit substantial gains while others quickly plateau. For instance, we find that Qwen-2.5-3B far exceeds Llama-3.2-3B under identical RL training for the game of Countdown. This discrepancy raises a critical question: what intrinsic properties enable effective self-improvement? We introduce a framework to investigate this question by analyzing four key *cognitive behaviors* — verification, backtracking, subgoal setting, and backward chaining — that both expert human problem solvers and successful language models employ. Our study reveals that Qwen naturally exhibits these reasoning behaviors, whereas Llama initially lacks them. In systematic experimentation with controlled behavioral datasets, we find that priming Llama with examples containing these reasoning behaviors enables substantial improvements during RL, matching or exceeding Qwen's performance. Importantly, the presence of reasoning behaviors, rather than correctness of answers, proves to be the critical factor — models primed with incorrect solutions containing proper reasoning patterns achieve comparable performance to those trained on correct solutions. Finally, leveraging continued pretraining with OpenWebMath data, filtered to amplify reasoning behaviors, enables the Llama model to match Qwen's self-improvement trajectory. Our findings establish a fundamental relationship between initial reasoning behaviors and the capacity for improvement, explaining why some language models effectively utilize additional computation while others plateau.[1]

"The limits of my language mean the limits of my world."
—Wittgenstein

1 Introduction

When humans encounter a difficult but solvable problem, we spend more time thinking deeply and deliberately to arrive at a solution. Remarkably, recent language models have begun demonstrating similar reasoning behaviors when trained to self-improve via reinforcement learning (Guo et al., 2025; Jaech et al., 2024). Training language models with reinforcement learning (RL) on verifiable problems is not a new approach (Zelikman et al., 2022; Havrilla et al., 2024; Hoffman et al., 2023), but older methods leveraging RL plateaued after a few iterations without exploring many effective ways to use test-time compute for thinking. In this work, we investigate the reasons behind this change in self-improvement capabilities, focusing on the presence of key cognitive behaviors in base language models.

We focus our investigation on two base models, Qwen-2.5-3B (Qwen et al., 2025) and Llama-3.2-3B (Grattafiori et al., 2024), which show striking differences when trained with

[1]Code available at https://github.com/kanishkg/cognitive-behaviors

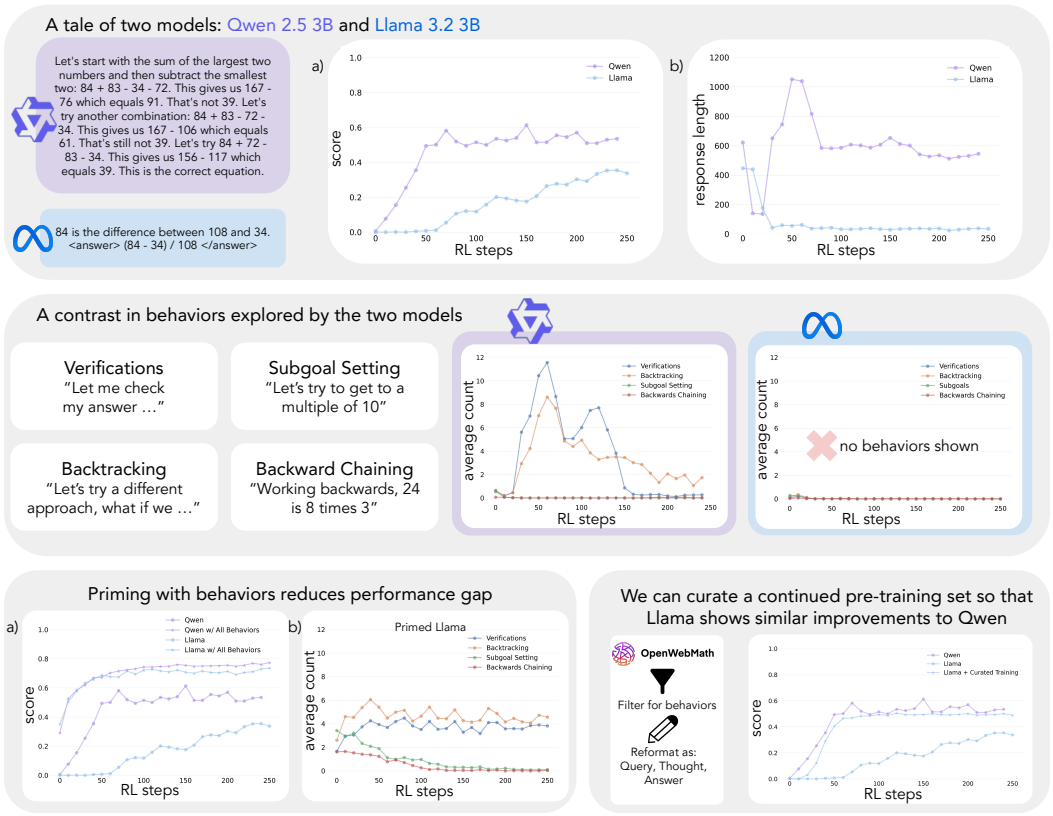


Figure 1: **Comparative analysis of Qwen-2.5-3B and Llama-3.2-3B models with RL on Countdown.** (Top) (a) Performance scores on Countdown for both models (b) Evolution of response lengths throughout RL training. (Middle) Emergence of specific reasoning characteristics as a function of training steps for Qwen-2.5-3B (left) and Llama-3.2-3B (right). (Bottom-Left) (a) Countdown performance when base models are primed with a synthetic dataset of desired reasoning behaviors; (b) Differential impact of RL on reasoning behaviors of the primed Llama3.2-3B: amplification of backtracking and verification contrasted with suppression of backward chaining and subgoal setting. (Bottom-Right) Comparative efficacy of teaching reasoning behaviors through fine-tuning on a curated OpenWebMath dataset, demonstrating that Llama's reasoning capabilities can be improved to match Qwen's through targeted training.

reinforcement learning on the game of Countdown (Countdown, 2024; Gandhi et al., 2024). While Qwen demonstrates substantial improvements in problem-solving ability, Llama[2] shows limited gains under an identical training process. What properties of the initial language model enable such improvements?

To systematically investigate this question, we develop a framework for analyzing *cognitive behaviors* that are useful for solving problems. We characterize four key cognitive behaviors: **verification** (systematic error-checking), **backtracking** (abandoning failing approaches), **subgoal setting** (decomposing problems into manageable steps), and **backward chaining** (reasoning from desired outcomes to initial inputs). These behaviors mirror how expert problem solvers approach difficult tasks — a mathematician verifies each step of a proof, backtracks when encountering contradictions, and breaks complex theorems into simpler lemmas. We examine these four behaviors because they can be used to represent search-based reasoning that goes beyond the typical "linear" reasoning shown by language models.

[2]For the remainder of this paper, we refer to Qwen2.5-3B as "Qwen" and Llama3.2-3B as "Llama", though full model names are also occasionally used. All other models are specified with complete designations.

While many other cognitive behaviors exist, we begin with these as they are well-defined and easily identifiable in model outputs.

Our initial analysis reveals that Qwen naturally exhibits these reasoning behaviors, particularly verification and backtracking, while Llama lacks them. This observation motivates our central hypothesis: certain reasoning behaviors in the initial policy are necessary for efficiently utilizing increased test-time compute through extended reasoning sequences. We test this hypothesis through interventions on the initial model. First, we demonstrate that priming Llama with synthetic reasoning traces containing these behaviors, especially backtracking, enables substantial improvements during RL, matching Qwen's performance trajectory. Second, these gains persist even when primed on incorrect solutions, if they exhibit proper reasoning patterns, suggesting that the presence of reasoning behaviors, rather than access to correct solutions, is the critical factor enabling successful self-improvement. Third, by curating pretraining data from OpenWebMath (Paster et al., 2023) that emphasizes these reasoning behaviors, we show that targeted modification of the pretraining distribution can successfully induce the behavioral patterns necessary for efficient use of test-time compute — the improvement trajectory of Llama matches that of Qwen.

Our investigation reveals a strong relationship between a model's initial reasoning behaviors and its capacity for improvement. This connection helps explain why some language models discover effective ways to use additional compute while others plateau. Understanding these dynamics may be key to developing AI systems that can meaningfully improve their problem-solving abilities.

2 Related Work

Recent approaches to improving reasoning capabilities in language models can be broadly categorized into three complementary directions: external search methods that leverage multiple samples, in-context search that enables models to reason over their own outputs, and reinforcement learning approaches that allow models to discover reasoning strategies autonomously.

External Search for Reasoning. Recent work has shown that language models can significantly improve their performance on complex tasks when given additional inference-time compute. Snell et al. (2024) systematically explore this space by developing various methods to search through reasoning trajectories. These approaches range from simple parallel sampling (Li et al., 2024; Brown et al., 2024) to more sophisticated methods using verifiers or process reward models (PRMs) (Lightman et al., 2023; Yao et al., 2023). Some researchers have taken this further by using the search process itself to improve the underlying reasoning model (Wang et al., 2023; Luo et al., 2024). However, these methods typically operate without awareness of previously explored solutions, limiting their efficiency through redundant exploration.

In-Context Search and Self-Improvement. In contrast to external search methods, another line of research focuses on enabling models to search sequentially in language. This has been achieved through various approaches such as 1) in-context examples (Gandhi et al., 2023), 2) finetuning on linearized search traces (Gandhi et al., 2024; Lehnert et al., 2024), and 3) training on self-correction examples (Ye et al., 2024; Qu et al., 2025; Kumar et al., 2024; Hwang et al., 2024). Recent work by Schultz et al. (2024) bridges the gap between in-context and external search methods, demonstrating improved performance on strategic games (cf. Xiang et al., 2025). While effective, these approaches often require careful engineering of training data to incorporate desired behaviors like self-correction and backtracking.

Reinforcement Learning for Reasoning. The prospect of models autonomously discovering effective reasoning strategies has motivated significant research in reinforcement learning approaches. Early work in teaching language models to reason with verifiable outcomes explored various RL methodologies, from off-policy and batch methods (Zelikman et al., 2022; Havrilla et al., 2024; Hoffman et al., 2023) to on-policy approaches (Zelikman et al., 2024; Kazemnejad et al., 2024; Cui et al., 2025). These methods differ in their approaches to credit assignment in reasoning trajectories. A notable breakthrough

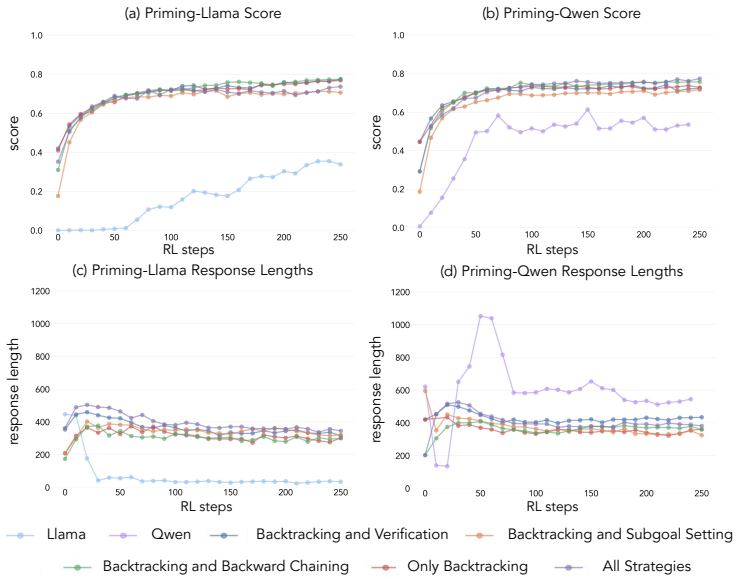


Figure 2: **The effects of priming with different cognitive behaviors.** (a, b) Performance comparison on the Countdown task between Llama-3.2-3B, Qwen-2.5-3B and their primed variants, illustrating the influence of reasoning behavior priming on scores. (c, d) Response length analysis for standard and primed Llama and Qwen models, showing how priming influences reasoning length.

came with Deepseek-R1 (Guo et al., 2025; Jaech et al., 2024), which demonstrated that even a simplified version of PPO (Schulman et al., 2017) called GRPO (Shao et al., 2024) could lead to significant improvements and the emergence of in-context search behaviors. Recent analyses (Li et al., 2025), Wu et al. (2024), Liu et al. (2025), Ye et al. (2025) and Yeo et al. (2025) have begun to unpack these results, revealing that supervised fine-tuning with long, structured chains of thought enhances both the efficiency and performance of RL compared to shorter reasoning chains. However, a crucial question remains unanswered: why do some models successfully learn through RL while others fail to improve? (Yeo et al., 2025)briefly discuss this issue by analyzing the frequency of phrases related to reflective reasoning in the pretraining dataset but their analysis remains limited. Our work addresses this gap by investigating the essential properties of initial models that enable successful reinforcement learning of reasoning behaviors.

3 Identifying and Engineering Self-Improving Behavior

3.1 Initial Investigation: A tale of two models

We begin by investigating a surprising observation: language models of comparable size but from different families show markedly different capacities for improvement through reinforcement learning. The Countdown game serves as our primary testbed — a mathematical puzzle where players must combine a set of input numbers using the four basic arithmetic operations ($+, -, \times, \div$) to reach a target number. For example, given the numbers 25, 30, 3, 4 and a target of 32, the solution involves combining these numbers through a series of operations to reach exactly 32: $(30 - 25 + 3) \times 4$.

We selected Countdown for our analysis because it demands mathematical reasoning, planning, and search strategies that mirror key aspects of general problem-solving similar to other reasoning domains such as math. Unlike more complex domains, Countdown offers a restricted search space that enables tractable analysis while still requiring sophisticated reasoning. Additionally, success in the game depends more on problem-solving abilities than mathematical knowledge, compared to other mathematical tasks where domain knowledge might confound the evaluation of reasoning capabilities.

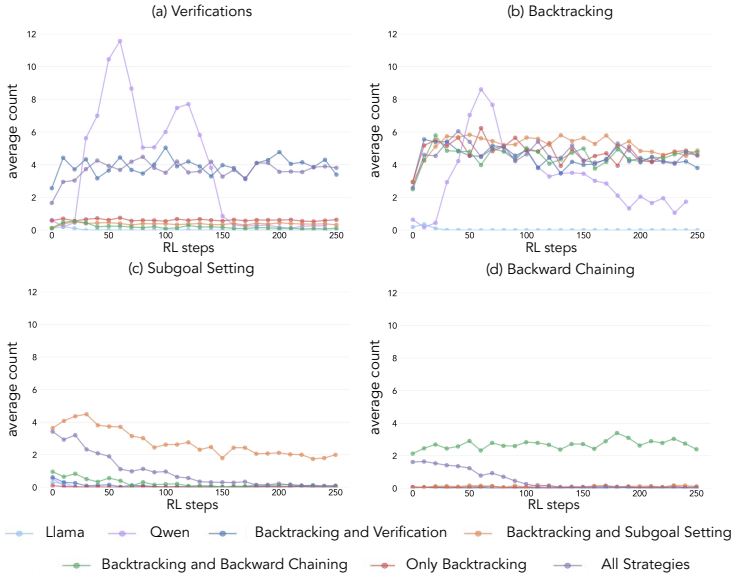


Figure 3: **Analysis of four key reasoning behaviors with Llama-3.2-3B, Qwen-2.5-3B, and primed versions of Llama-3.2-3B.** Plots show mean frequency of (a) solution verification steps, (b) problem-solving backtracking instances, (c) explicit subgoal setting, and (d) backward chaining reasoning approaches across different tasks.

We use two base models to contrast how learning varies across model families: Qwen-2.5-3B and Llama-3.2-3B. Our reinforcement learning experiments build on the VERL library (Sheng et al., 2024), utilizing the TinyZero (Pan et al., 2025) implementation. We train models with PPO (Schulman et al., 2017) for 250 steps, sampling 4 trajectories per prompt. We chose PPO over alternatives like GRPO (Shao et al., 2024) and REINFORCE (Williams, 1992; Hu, 2025; Ahmadian et al., 2024) as it demonstrated superior stability across hyperparameter settings, though performance was anecdotally similar across algorithms.

The results reveal strikingly different learning trajectories. Though both models start at a similar, low performance on the task, Qwen demonstrates a qualitative shift around step 30, characterized by significantly longer responses and improved accuracy (Fig. 1 (top)). By the end of training, Qwen achieves approximately 60% accuracy, substantially outperforming Llama's 30%. Later in training, we observe an interesting change in Qwen's behavior: the model transitions from explicit verification statements in language, "8*35 is 280 which is too high" to implicit solution checking, where the model sequentially tries different solutions until it finds the right answer without using words to evaluate its own work.

This contrast raises a fundamental question: what underlying capabilities enable successful reasoning-based improvement? To answer this, we need a systematic framework for analyzing cognitive behaviors.

3.2 A Framework for Analyzing Cognitive Behaviors

To understand these divergent learning trajectories, we develop a framework for identifying and analyzing key behaviors in model outputs. We focus on four fundamental behaviors: (1) Backtracking or the explicit revision of approaches when errors are detected (e.g., "This approach won't work because..."), (2) Verification or the systematic checking of intermediate results (e.g., "Let's verify this result by..."), (3) Subgoal Setting, where a complex problem is broken down into manageable steps (e.g., "To solve this, we first need to..."), and (4) Backward Chaining, where in a goal-directed reasoning problem, the solution works backwards from a desired outcomes (e.g., "To reach the target of 75, we need a number divisible by...").

We selected these behaviors because they represent problem-solving strategies that deviate from the linear, monotonic reasoning patterns commonly observed in language models. These behaviors enable more dynamic, search-like reasoning trajectories where solutions

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei **Xuezhi Wang** **Dale Schuurmans** **Maarten Bosma**

Brian Ichter **Fei Xia** **Ed H. Chi** **Quoc V. Le** **Denny Zhou**

Google Research, Brain Team
{jasonwei,dennyzhou}@google.com

Abstract

We explore how generating a *chain of thought*—a series of intermediate reasoning steps—significantly improves the ability of large language models to perform complex reasoning. In particular, we show how such reasoning abilities emerge naturally in sufficiently large language models via a simple method called *chain-of-thought prompting*, where a few chain of thought demonstrations are provided as exemplars in prompting.

Experiments on three large language models show that chain-of-thought prompting improves performance on a range of arithmetic, commonsense, and symbolic reasoning tasks. The empirical gains can be striking. For instance, prompting a PaLM 540B with just eight chain-of-thought exemplars achieves state-of-the-art accuracy on the GSM8K benchmark of math word problems, surpassing even finetuned GPT-3 with a verifier.

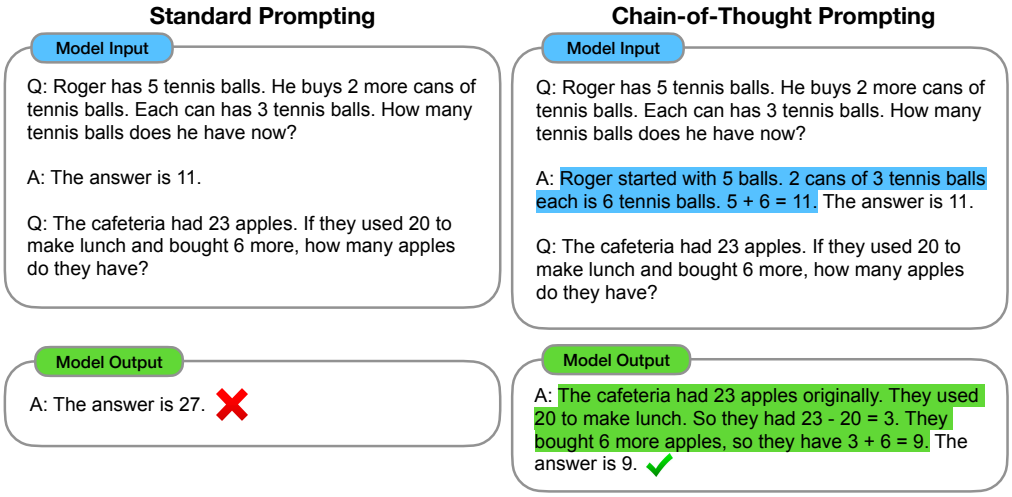


Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

36th Conference on Neural Information Processing Systems (NeurIPS 2022).

1 Introduction

The NLP landscape has recently been revolutionized by language models (Peters et al., 2018; Devlin et al., 2019; Brown et al., 2020, *inter alia*). Scaling up the size of language models has been shown to confer a range of benefits, such as improved performance and sample efficiency (Kaplan et al., 2020; Brown et al., 2020, *inter alia*). However, scaling up model size alone has not proved sufficient for achieving high performance on challenging tasks such as arithmetic, commonsense, and symbolic reasoning (Rae et al., 2021).

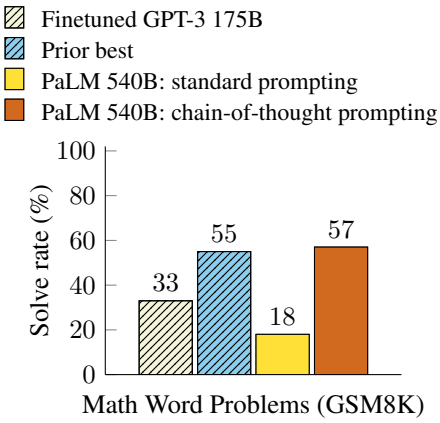


Figure 2: PaLM 540B uses chain-of-thought prompting to achieve new state-of-the-art performance on the GSM8K benchmark of math word problems. Finetuned GPT-3 and prior best are from Cobbe et al. (2021).

This work explores how the reasoning ability of large language models can be unlocked by a simple method motivated by two ideas. First, techniques for arithmetic reasoning can benefit from generating natural language rationales that lead to the final answer. Prior work has given models the ability to generate natural language intermediate steps by training from scratch (Ling et al., 2017) or finetuning a pretrained model (Cobbe et al., 2021), in addition to neuro-symbolic methods that use formal languages instead of natural language (Roy and Roth, 2015; Chiang and Chen, 2019; Amini et al., 2019; Chen et al., 2019). Second, large language models offer the exciting prospect of in-context few-shot learning via *prompting*. That is, instead of finetuning a separate language model checkpoint for each new task, one can simply "prompt" the model with a few input–output exemplars demonstrating the task. Remarkably, this has been successful for a range of simple question-answering tasks (Brown et al., 2020).

Both of the above ideas, however, have key limitations. For rationale-augmented training and finetuning methods, it is costly to create a large set of high quality rationales, which is much more complicated than simple input–output pairs used in normal machine learning. For the traditional few-shot prompting method used in Brown et al. (2020), it works poorly on tasks that require reasoning abilities, and often does not improve substantially with increasing language model scale (Rae et al., 2021). In this paper, we combine the strengths of these two ideas in a way that avoids their limitations. Specifically, we explore the ability of language models to perform few-shot prompting for reasoning tasks, given a prompt that consists of triples: ⟨input, *chain of thought*, output⟩. A *chain of thought* is a series of intermediate natural language reasoning steps that lead to the final output, and we refer to this approach as *chain-of-thought prompting*. An example prompt is shown in Figure 1.

We present empirical evaluations on arithmetic, commonsense, and symbolic reasoning benchmarks, showing that chain-of-thought prompting outperforms standard prompting, sometimes to a striking degree. Figure 2 illustrates one such result—on the GSM8K benchmark of math word problems (Cobbe et al., 2021), chain-of-thought prompting with PaLM 540B outperforms standard prompting by a large margin and achieves new state-of-the-art performance. A prompting only approach is important because it does not require a large training dataset and because a single model checkpoint can perform many tasks without loss of generality. This work underscores how large language models can learn via a few examples with natural language data about the task (c.f. automatically learning the patterns underlying inputs and outputs via a large training dataset).

2 Chain-of-Thought Prompting

Consider one's own thought process when solving a complicated reasoning task such as a multi-step math word problem. It is typical to decompose the problem into intermediate steps and solve each before giving the final answer: *"After Jane gives 2 flowers to her mom she has 10 . . . then after she gives 3 to her dad she will have 7 . . . so the answer is 7."* The goal of this paper is to endow language models with the ability to generate a similar *chain of thought*—a coherent series of intermediate reasoning steps that lead to the final answer for a problem. We will show that sufficiently large

language models can generate chains of thought if demonstrations of chain-of-thought reasoning are provided in the exemplars for few-shot prompting.

Figure 1 shows an example of a model producing a chain of thought to solve a math word problem that it would have otherwise gotten incorrect. The chain of thought in this case resembles a solution and can interpreted as one, but we still opt to call it a chain of thought to better capture the idea that it mimics a step-by-step thought process for arriving at the answer (and also, solutions/explanations typically come *after* the final answer (Narang et al., 2020; Wiegreffe et al., 2022; Lampinen et al., 2022, *inter alia*)).

Chain-of-thought prompting has several attractive properties as an approach for facilitating reasoning in language models.

1. First, chain of thought, in principle, allows models to decompose multi-step problems into intermediate steps, which means that additional computation can be allocated to problems that require more reasoning steps.
2. Second, a chain of thought provides an interpretable window into the behavior of the model, suggesting how it might have arrived at a particular answer and providing opportunities to debug where the reasoning path went wrong (although fully characterizing a model's computations that support an answer remains an open question).
3. Third, chain-of-thought reasoning can be used for tasks such as math word problems, commonsense reasoning, and symbolic manipulation, and is potentially applicable (at least in principle) to any task that humans can solve via language.
4. Finally, chain-of-thought reasoning can be readily elicited in sufficiently large off-the-shelf language models simply by including examples of chain of thought sequences into the exemplars of few-shot prompting.

In empirical experiments, we will observe the utility of chain-of-thought prompting for arithmetic reasoning (Section 3), commonsense reasoning (Section 4), and symbolic reasoning (Section 5).

3 Arithmetic Reasoning

We begin by considering math word problems of the form in Figure 1, which measure the arithmetic reasoning ability of language models. Though simple for humans, arithmetic reasoning is a task where language models often struggle (Hendrycks et al., 2021; Patel et al., 2021, *inter alia*). Strikingly, chain-of-thought prompting when used with the 540B parameter language model performs comparably with task-specific finetuned models on several tasks, even achieving new state of the art on the challenging GSM8K benchmark (Cobbe et al., 2021).

3.1 Experimental Setup

We explore chain-of-thought prompting for various language models on multiple benchmarks.

Benchmarks. We consider the following five math word problem benchmarks: **(1)** the **GSM8K** benchmark of math word problems (Cobbe et al., 2021), **(2)** the **SVAMP** dataset of math word problems with varying structures (Patel et al., 2021), **(3)** the **ASDiv** dataset of diverse math word problems (Miao et al., 2020), **(4)** the **AQuA** dataset of algebraic word problems, and **(5)** the **MAWPS** benchmark (Koncel-Kedziorski et al., 2016). Example problems are given in Appendix Table 12.

Standard prompting. For the baseline, we consider standard few-shot prompting, popularized by Brown et al. (2020), in which a language model is given in-context exemplars of input–output pairs before outputting a prediction for a test-time example. Exemplars are formatted as questions and answers. The model gives the answer directly, as shown in Figure 1 (left).

Chain-of-thought prompting. Our proposed approach is to augment each exemplar in few-shot prompting with a chain of thought for an associated answer, as illustrated in Figure 1 (right). As most of the datasets only have an evaluation split, we manually composed a set of eight few-shot exemplars with chains of thought for prompting—Figure 1 (right) shows one chain of thought exemplar, and the full set of exemplars is given in Appendix Table 20. (These particular exemplars did not undergo prompt engineering; robustness is studied in Section 3.4 and Appendix A.2.) To investigate whether chain-of-thought prompting in this form can successfully elicit successful reasoning across a range of

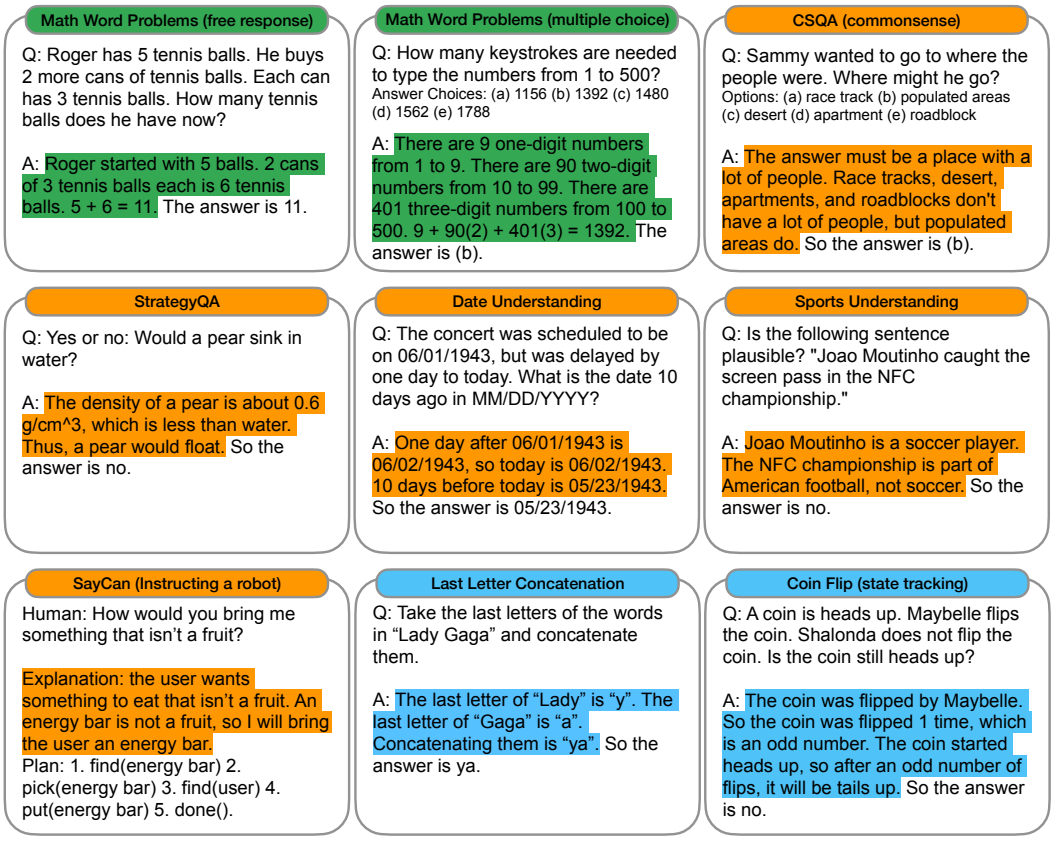


Figure 3: Examples of ⟨input, chain of thought, output⟩ triples for arithmetic, commonsense, and symbolic reasoning benchmarks. Chains of thought are highlighted. Full prompts in Appendix G.

math word problems, we used this single set of eight chain of thought exemplars for all benchmarks except AQuA, which is multiple choice instead of free response. For AQuA, we used four exemplars and solutions from the training set, as given in Appendix Table 21.

Language models. We evaluate five large language models. The first is **GPT-3** (Brown et al., 2020), for which we use text-ada-001, text-babbage-001, text-curie-001, and text-davinci-002, which presumably correspond to InstructGPT models of 350M, 1.3B, 6.7B, and 175B parameters (Ouyang et al., 2022).The second is **LaMDA** (Thoppilan et al., 2022), which has models of 422M, 2B, 8B, 68B, and 137B parameters. The third is **PaLM**, which has models of 8B, 62B, and 540B parameters. The fourth is **UL2 20B** (Tay et al., 2022), and the fifth is **Codex** (Chen et al., 2021, code-davinci-002 in the OpenAI API). We sample from the models via greedy decoding (though follow-up work shows chain-of-thought prompting can be improved by taking the majority final answer over many sampled generations (Wang et al., 2022a)). For LaMDA, we report averaged results over five random seeds, where each seed had a different randomly shuffled order of exemplars. As LaMDA experiments did not show large variance among different seeds, to save compute we report results for a single exemplar order for all other models.

3.2 Results

The strongest results of chain-of-thought prompting are summarized in Figure 4, with all experimental outputs for each model collection, model size, and benchmark shown in Table 2 in the Appendix. There are three key takeaways. First, Figure 4 shows that chain-of-thought prompting is an emergent ability of model scale (Wei et al., 2022b). That is, chain-of-thought prompting does not positively impact performance for small models, and only yields performance gains when used with models of $\sim$100B parameters. We qualitatively found that models of smaller scale produced fluent but illogical chains of thought, leading to lower performance than standard prompting.

Second, chain-of-thought prompting has larger performance gains for more-complicated problems. For instance, for GSM8K (the dataset with the lowest baseline performance), performance more than doubled for the largest GPT and PaLM models. On the other hand, for SingleOp, the easiest subset of MAWPS which only requires a single step to solve, performance improvements were either negative or very small (see Appendix Table 3).

Third, chain-of-thought prompting via GPT-3 175B and PaLM 540B compares favorably to prior state of the art, which typically finetunes a task-specific model on a labeled training dataset. Figure 4 shows how PaLM 540B uses chain-of-thought prompting to achieve new state of the art on GSM8K, SVAMP, and MAWPS (though note that standard prompting already passed the prior best for SVAMP). On the other two datasets, AQuA and ASDiv, PaLM with chain-of-thought prompting reaches within 2% of the state of the art (Appendix Table 2).

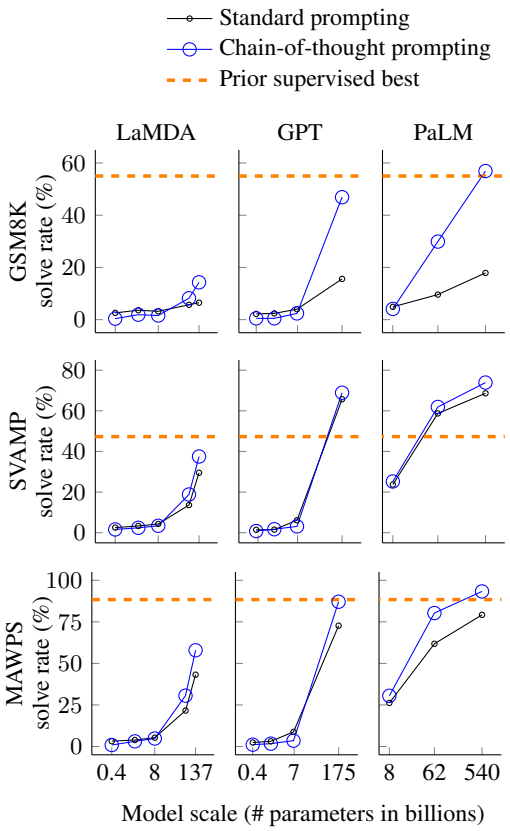


Figure 4: Chain-of-thought prompting enables large language models to solve challenging math problems. Notably, chain-of-thought reasoning is an emergent ability of increasing model scale. Prior best numbers are from Cobbe et al. (2021) for GSM8K, Jie et al. (2022) for SVAMP, and Lan et al. (2021) for MAWPS.

To better understand why chain-of-thought prompting works, we manually examined model-generated chains of thought by LaMDA 137B for GSM8K. Of 50 random examples where the model returned the correct final answer, all of the generated chains of thought were also logically and mathematically correct except two that coincidentally arrived at the correct answer (see Appendix D.1, and Table 8 for examples of correct model-generated chains of thought). We also randomly examined 50 random samples for which the model gave the wrong answer. The summary of this analysis is that 46% of the chains of thought were almost correct, barring minor mistakes (calculator error, symbol mapping error, or one reasoning step missing), and that the other 54% of the chains of thought had major errors in semantic understanding or coherence (see Appendix D.2). To provide a small insight into why scaling improves chain-of-thought reasoning ability, we performed a similar analysis of errors made by PaLM 62B and whether those errors were fixed by scaling to PaLM 540B. The summary is that scaling PaLM to 540B fixes a large portion of one-step missing and semantic understanding errors in the 62B model (see Appendix A.1).

3.3 Ablation Study

The observed benefits of using chain-of-thought prompting raises the natural question of whether the same performance improvements can be conferred via other types of prompting. Figure 5 shows an ablation study with three variations of chain of thought described below.

Equation only. One reason for why chain-of-thought prompting might help is that it produces the mathematical equation to be evaluated, and so we test a variation where the model is prompted to output only a mathematical equation before giving the answer. Figure 5 shows that equation only prompting does not help much for GSM8K, which implies that the semantics of the questions in GSM8K are too challenging to directly translate into an equation without the natural language reasoning steps in chain of thought. For datasets of one-step or two-step problems, however, we find that equation only prompting does improve performance, since the equation can be easily derived from the question (see Appendix Table 6).